

Service Discovery

 systemdesignschool.io/fundamentals/service-discovery



We explored how to implement synchronous communication and handle failures. But a fundamental question remains: how does Service A know where to find Service B? In a dynamic microservices environment, service instances start and stop constantly. IP addresses change. Auto-scaling adds and removes servers. Hardcoding addresses does not work.

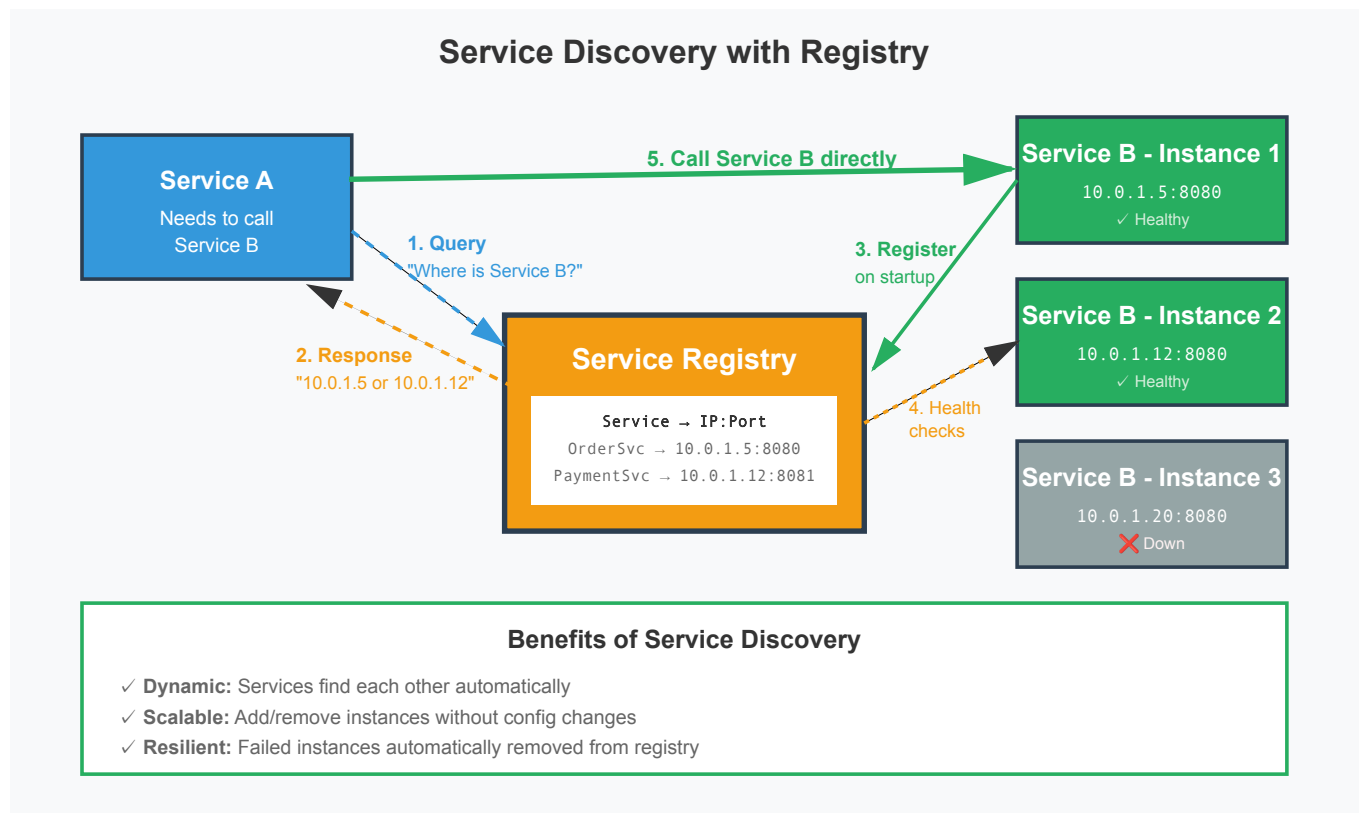
Why Service Discovery Matters

Service A needs to call Service B. In a monolithic application, both components run in the same process. A function call suffices. In microservices, they run on different machines. Service A needs Service B's IP address and port to send HTTP or gRPC requests.

Hardcoding IP addresses in configuration files breaks when services move or scale. If Service B runs on 10.0.1.5 today but auto-scaling launches a new instance at 10.0.1.12 tomorrow, Service A cannot find it. Manual configuration updates do not scale.

Service discovery solves this by maintaining a registry. Each service registers its location when it starts. Services query the registry to find dependencies. When instances scale up or down, the registry updates automatically. Service A always finds Service B without manual intervention.

How Service Discovery Works



A service registry is a database of service locations. When Service B starts, it registers itself with the registry. The registration includes the service name, IP address, port, and health check endpoint.

Service A needs to call Service B. It queries the registry: "Where is Service B?" The registry returns a list of available instances: 10.0.1.5:8080 and 10.0.1.12:8080. Service A picks one and makes the request.

The registry performs health checks on registered instances. Every 10 seconds, it pings each service's health endpoint. If a service fails three consecutive health checks, the registry removes it from the list. When Service A queries next time, it only gets healthy instances.

When a new Service B instance starts at 10.0.1.20:8080, it registers immediately. The registry adds it to the list. Service A's next query includes all three instances. Load balances across them automatically.

Client-Side Discovery

In client-side discovery, the client queries the registry directly. Service A calls the registry, gets a list of Service B instances, and picks one. The client handles load balancing by choosing different instances for each request.

Netflix Eureka uses this pattern. Each service maintains a local cache of the registry. The cache refreshes every 30 seconds. When Service A needs Service B, it checks its local cache and selects an instance. This is fast because no network call to the registry is needed per request.

Client-side discovery gives clients control. They can implement custom load balancing strategies. Route more traffic to faster instances. Pin certain users to specific servers for cache locality. But it adds complexity. Every client needs discovery and load balancing logic.

Server-Side Discovery

In server-side discovery, clients call a load balancer. The load balancer queries the registry and forwards the request to an appropriate instance. Service A sends requests to `loadbalancer.example.com`. The load balancer looks up Service B instances in the registry and proxies the request.

Kubernetes uses this pattern. A Service object acts as the load balancer. Clients call the service by name. Kubernetes DNS resolves the name to the service IP. The service forwards requests to healthy pods based on the registry.

Server-side discovery keeps clients simple. They just call a fixed address. The load balancer handles discovery and routing. But the load balancer becomes a potential bottleneck and single point of failure.

Platform-Provided Discovery

Container orchestration platforms like Kubernetes and Cloud Foundry provide built-in service discovery. Deploy a service called "inventory" and Kubernetes creates a DNS entry. Other services call `"inventory.default.svc.cluster.local"` and Kubernetes routes to a healthy pod.

This integration is convenient. No separate registry to manage. Discovery is automatic. But it ties you to the platform. Moving off Kubernetes means implementing discovery differently.

Service Registry Implementations

Consul is a popular standalone registry. It provides service registration, health checking, and a key-value store for configuration. Services register via HTTP API. Consul performs TCP or HTTP health checks. Clients query via DNS or HTTP.

Zookeeper is another option. It maintains a tree of service nodes. Each service creates an ephemeral node when it starts. If the service crashes, Zookeeper deletes the node automatically. Clients watch for changes and update their routing tables.

Cloud providers offer managed solutions. AWS Cloud Map integrates with EC2 and ECS. Google Cloud Service Directory works with GKE and Compute Engine. These handle registration and health checking automatically for platform services.

Health Checks

Health checks ensure the registry only returns working instances. A basic health check pings an HTTP endpoint. If it returns 200 OK, the service is healthy. If it times out or returns an error, mark it unhealthy.

Deep health checks verify dependencies. The inventory service checks its database connection. If the database is unreachable, return 503 Service Unavailable. The registry marks the instance unhealthy even though the service process is running.

Passive health checks detect failures through traffic. A load balancer tracks request success rates. If 10% of requests to an instance fail, mark it unhealthy. This catches problems that periodic health checks might miss.

Service discovery is essential infrastructure for microservices. It enables dynamic scaling and deployment without manual configuration. Combined with health checking and load balancing, it routes traffic only to healthy instances and adapts automatically as services scale.